摘要：

每月烧掉千万美金的“算力特权”，是软件工程的未来，还是制造 AI 垃圾代码的公关秀？

10 月 21 日，OpenClaw 创始人 Peter Steinberger 在 Cloudflare Connect 的台上估算，自己上个月用于 AI 工作的 token 成本约为 1000 万美元。听众发出惊呼，他补了一句：“并非所有消耗都有用。”

几周前，另一位 Peter——Peter Piekarczyk——也算出了一笔让人咋舌的账。他看到 Lauren Tan 的 Cursor 个人主页显示，过去 30 天用了 1.5 万亿 token，于是在 X 上写道：“按每百万 token 8 美元估算，Lauren 每天消耗约 38.4 万美元，年化 1.32 亿美元。”即使假设 Cursor 能拿到大客户一折优惠，年成本仍有约 1300 万美元。

这组数字很快引发了大量关注，因为它把看不见的 token 换成了人人都能理解的工资单。在算账之前，先得知道 1.5 万亿有多夸张。工程管理平台 Jellyfish 对 1.2 万名开发者的统计显示，典型用户（第 50 百分位）每月约使用 5100 万 token，第 90 百分位用户约为 3.8 亿。粗略对照，Lauren 的用量约为典型用户的 2.9 万倍；Peter Steinberger 5 月公开的 6030 亿 token，则约为 1.2 万倍。

这笔“账单”是怎么算出来的？

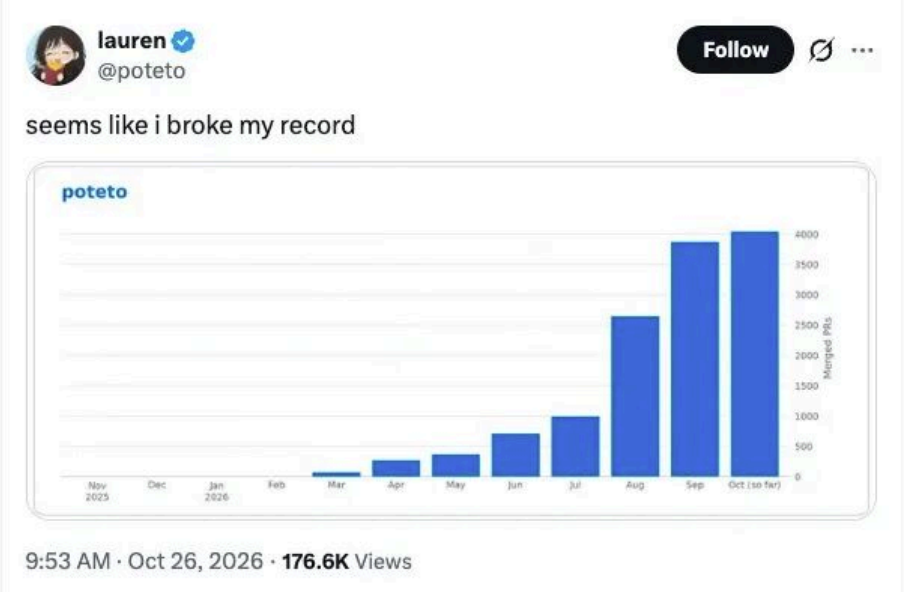
Lauren Tan 目前在 SpaceXAI Grok 团队担任工程师。她的职业跨度不小：大学学的是金融，最初在 Excel 里写 VBA 宏时接触编程，职业生涯从 UI 设计起步，后来转向前端和 React，参与 React Compiler 项目；AI 浪潮起来后，她又把重心转向 Agent 工作流，做出了 pstack，一套智能体 skills。

她也很懂得经营自己的技术形象。9 月中旬，她在一次视频演讲中说自己“每月向生产环境合入 2000 个 PR”——这意味着平均每个工作日就能提交近 100 个 PR。这个数字令人难以置信，无疑是个异常值。

9 月 21 日，她自己在 X 上置顶发布了这个“2000 个 PR”的视频，但配文却变成“上个月我向生产环境合入了 2500 个 PR”，引发了新一轮的讨论。



10 月 26 日，她再次发帖说“好像破纪录了”，配图是 4000 个 PR。数字一次比一次高，每一次都配上公开发布。



此前，她的 Cursor 个人主页显示，在 9 月份的 30 天里使用了约 1.5 万亿个 token。

营销这件事本身不是罪，但当你把一个极端值放到公共橱窗里，就得准备好它被别人拿去乘一个单价。

Peter Piekarczyk 就把这个公开数字换算成了美元：平均每百万 token 8 美元估算，得出每天约 38.4 万美元、30 天约 1100 万美元、年化 1.32 亿美元的结果。按他的算法，这就是“一个工程师一年烧掉 1.32 亿美元”。

他接着假设 Cursor 能从 Anthropic 拿到九成折扣，也就是按原价一折结算，费用便降到每天约 3.8 万美元、30 天约 110 万美元，年化约 1300 万美元。再按资深工程师年薪 50 万美元计算，这笔钱相当于雇 26 名资深工程师。

考虑到她使用的模型可能有区别，如果全部使用 Grok，30 天约花 480 万美元；全部使用 Opus 或 Sonnet，约 1320 万美元；按 40% 输入、40% 输出、20% 缓存读取的混合用量估算，则约 1150 万美元。再套用不同折扣，混合用量的月成本还可能降到 580 万或 990 万美元。

Scenario	Effective $/1M	30-day cost	Per day
All Grok	$3.20	**$4.8M**	$160K
All Opus/Sol	$8.80	**$13.2M**	$440K
Everyday API blend (40/40/20)	$7.68	**$11.5M**	$384K
50% off that blend	$3.84	**$5.8M**	$192K
15% wholesale (Opus/Sol −15%, Grok list)	$6.62	**$9.9M**	$331K

Agent 编程还存在缓存命中的情况，假设缓存读取的价格约为普通输入价格的十分之一，每月仍然是 100 万美元。



不过，Piekarczyk 的算法依然还有争议。

Cursor 主页统计的 token 至少分四类：普通输入、模型输出、缓存写入、缓存读取，价格差异很大。以 Opus 5.5 为例，每百万普通输入约 4 美元，输出约 20 美元，缓存读取约 0.20 美元。

一张 Opus 5.5 的用量截图显示，733 亿 token 对应约 3 万美元，折合每百万 token 约 0.41 美元；其中缓存读取和写入合计占了大部分费用。如果把这个均价套到 Lauren 的 1.5 万亿 token 上，结果约为每月 61.5 万美元。



还有一层可以纳入考虑：Lauren 做的是深入、长时运行的智能体任务，比普通用户更常命中缓存。长时任务反复复用同一份上下文，这一项在总花费里的占比会被进一步推高。Anthropic 公布的 Opus 5.5 价格中，缓存读取为每百万 token 0.20 美元，普通输入为 4 美元。如果按 1.5 万亿 token 全部以缓存读取价计费，月费用约 30 万美元；这当然只是极端下限。

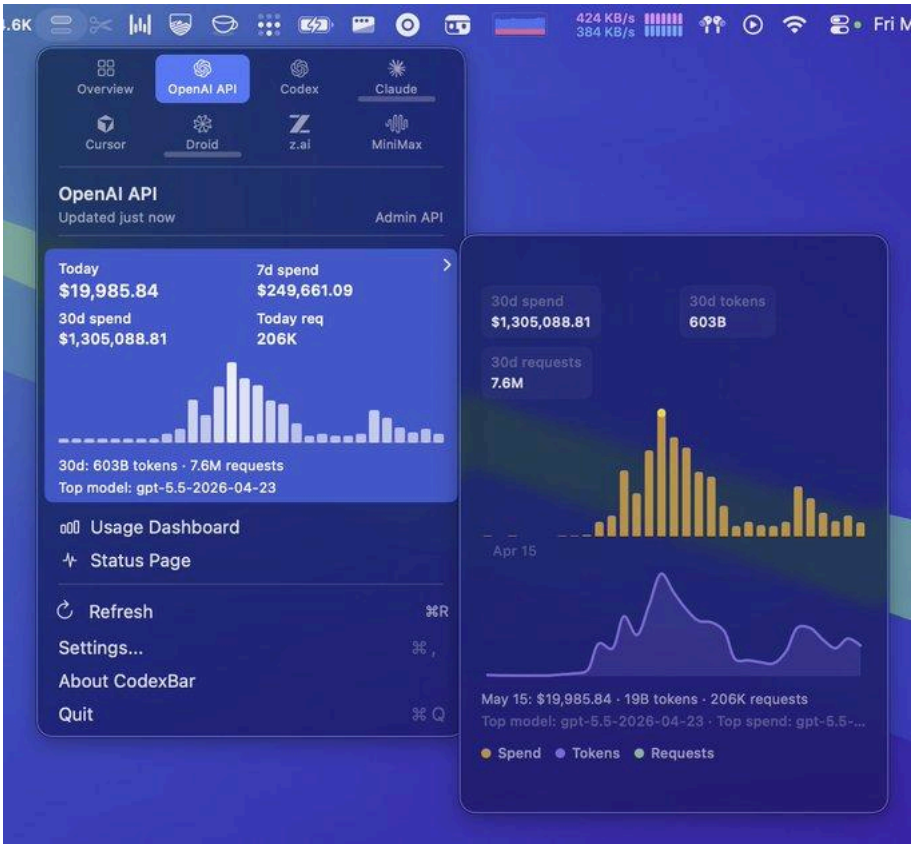
Piekarczyk 自己后来也退了一步：就算自己的算法高估了 10 倍，或者 Cursor 使用自有 GPU，那么按每百万 token 约 0.80 美元估算，月成本约为 10 万美元。

10 月 9 日，Piekarczyk 发起投票询问大家工作中每月的 AI 预算上限，大部分人每月低于 500 美元。和 Lauren 被估算出的月账单放在一起，反差鲜明。



另外作为对比，今年 5 月，Peter Steinberger 在 X 上也曾晒出 CodexBar 截图：过去 30 天 OpenAI API 用量约 130.5 万美元，涉及 6030 亿个 token、760 万次请求。后来他解释，截图按 Codex 的 Fast Mode 价格计算，关闭 Fast Mode 后估算的 API 成本约为 30 万美元。

10 月这次 1000 万美元是活动现场的口头估算，但有一点是一致的：它们的数量级，和普通开发者每月几百美元的预算完全是两个世界。



普通人不能尝试的工作方式

Steinberger 把这类投入描述为探索“如果 Token 成本不再构成限制，软件开发会变成什么样”。他的三人团队当时运行着约 100 个 Codex 实例。

这些 Agent 集群能够自主审查 Pull Request、扫描提交记录以查找安全漏洞、合并重复的 GitHub Issue，并编写修复代码。有些 Agent 会根据项目的整体路线图主动提交 PR；另一些则持续监控性能基准测试，并将性能回退情况报告到团队的 Discord 服务器。据 The Decoder 报道，有些 Agent 甚至会参加会议，并根据讨论中提到的功能生成 PR。

Lauren 每月 1.5 万亿 token，无论按什么口径计算，同样都是普通人不能承受的金额。那么这笔钱推动了什么？按她自己公开的说法是，她如今每月可以高置信度地把 4000 个 PR 合入生产环境。这个数字的关键不在生成，在验证。

“使用 Agent 时最重要的技能是验证。所谓验证，是指让 Agent 真正运行代码、采集 CPU 跟踪或堆快照、打开 iOS 模拟器，或者用用户接触应用的方式去操作应用，实际测试并确认功能是否正常。验证能把工作闭环。它不能保证 Agent 写出优秀代码，至少能帮助它写出正确的代码；这已经是建立信任的一大步。”

她的验证技能让 Agent 检查自己的工作并持续迭代，直到任务完成。逻辑是：能检查自身输出的 Agent 会一直工作到任务完成；不能的 Agent 只能把 diff 交给你然后停下——你就成了整个循环里最慢的环节。

按每月 4000 个 PR，若每个都由人花 5 分钟检查，累计约 333 小时，一个人的整月工时都不够用。人工审查不可能成为验证层，检查系统必须无人介入运行，并与生成代码的 Agent 并行。

在这个循环里，工程师的角色更像是一个“Agent 的工程经理”。Lauren 说，如果连一个 Agent 的输出都不信，就没法扩展到一百个 Agent。



她认为，绝大多数人根本没有把 Agent 用起来：“99% 的人只发挥了 GrokBot 1% 的真正实力。他们只运行一个 Agent，既没有‘循环（loop）’，也没有‘图（graph）’。我运行着一支由 20 多个 GrokBot Agent 组成的团队，全程自主运作。我有一个幕僚长 Agent、一个项目经理 Agent，还有 20 多个执行任务的 Agent——这就是工程师的新技术栈。”

她公开的 pstack 工作流，本质上就是这套多 Agent 编排的具体实现。通过分步的剧本引导（Playbook），将研发任务标准化为理解、设计、编码、评审到验证的顺序阶段。它通过在本地并发调度子智能体（如分析行为的 /how、审计风险的 /interrogate）来各司职守。

事实上，pstack 记录和提炼的，正是她从刚入职 Cursor、对老代码库一无所知开始，在本地通过观察 Agent 反复出现的失败模式而逐个堆叠出的垂直验证技能（如 Control Glass）。这种多 Agent 协作流虽然在云端消耗巨大，但由于模型在本地运行时缺乏无限的上下文，要让它真正高置信度地跑通，代码库本身也得适合 Agent 接手。

GrokBot 最初是一个快速 Vibe Coding 出来的新应用，人类几乎没有读过代码，架构约束也不充分。这种模式下，Agent 在执行任务时会优先选择眼前最省事的写法，导致代码越堆越多，功能边界和性能风险彻底失控。要想让 Agent 长期安全地接手开发，代码库就必须改造得容易被模型理解，修改路径必须清晰、且无可逃避地被验证。

为此，Lauren 用 600 多个 PR，把 GrokBot 重构到一套内部代号为 Dune 的架构中，一些 PR 比较微型，从几十行、几百行到一千行不等。在这种重构思路下，Dune 架构的 CI 检查极其严苛：全面禁用 useEffect 以消除隐式副作用；绝对禁止写注释，防止模型被包含过时历史信息的无用上下文误导而产生过度推断；同时严密校验依赖图，确保 Electron 的 main 和 renderer 目录绝不交叉导入。

当这套高密度的重构护栏架设完成时，由于智能体爱走捷径的本能被 CI 红线强行扭转，整个系统的生命周期管理彻底走向自动化。

在 Cursor 特定场景中，这套个人经验被打包成了 pstack，内部包含 23 个 Playbook（工作流剧本）和 20 多条工程原则。它是 Lauren 试图把特定项目的纪律塑造成 Agent 跑通测试的“唯一捷径”，在一些人口中被捧上了天，但这套高度定制化的教条在通用开发场景下对普通开发者究竟有多少落地价值，在其复杂的配置成本面前，显然得打上一个问号。

算力特权下的“AI Slop”工厂

这种依赖极致资源堆砌出来的交付模式，本质上是极少数人才能享有的资本特权。正如 Peter Steinberger 所直言的，“并非所有消耗都有用”，这种流派的盲目消耗往往只是在试错。当这种脱离了商业 ROI（投资回报率）的极端案例被推向公共橱窗时，Reddit 等技术社区的工程师们立刻发起了猛烈的声讨。

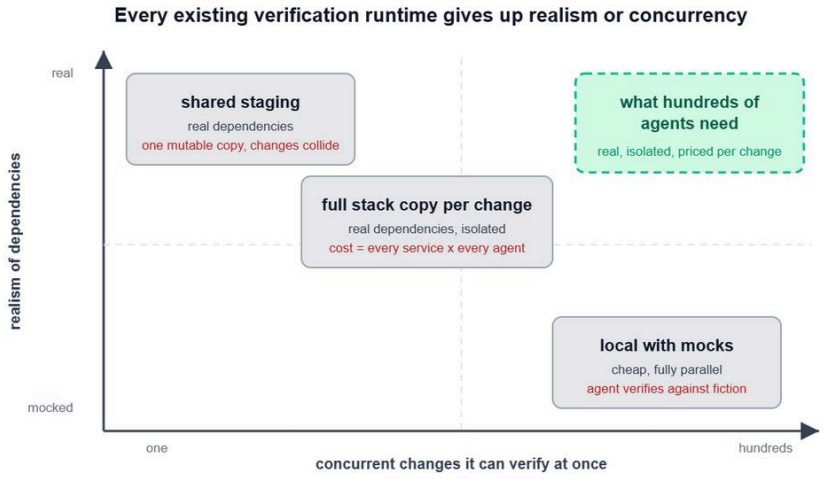
在 reddit 的热门讨论中，一线开发者们直言不讳地指出，这种每个月靠无限资金合入数千个 PR 的狂热表现，根本不是什么软件工程的底层革命，而是在用完全不计成本的暴力算力，不知吞吐地向代码库里灌入大规模的“AI Slop”（AI 垃圾内容）。



同行们的嘲讽直击这种暴力流派的软肋：软件开发真正的瓶颈从来不在于代码生成的“绝对数量”，而在于系统长期的可维护性。在普通工程师普遍低于 500 美元的月度 AI 预算面前，一旦剥去“Token 消耗不限量”的外衣，缺乏深思熟虑的微型 PR 就会彻底失控。

这种靠天价 Token 进行盲目试错和应激性防呆重构的流水线，看似缩短了交付路径，实则是将系统设计的主导权，从人类架构师的优雅全局思考，让渡给了机器在沙盒里的机械堆砌。这种

脱离群众的算力特权，最终留给生产环境的往往不是什么高效资产，而是让普通人在现实中根本无力维护的庞大技术债。



但在商业营销的逻辑里，这笔巨大的算力消耗在宣传上确实值了。T3 创始人 Theo 算过一笔账，他估算 Lauren 实际的“内部价”其实在每月 2.5 万到 5 万美元之间，并直言：“这类曝光的价值远不止每月 2.5 万美元；有些公司办活动花费超过 100 万美元，获得的关注还不如 Lauren 随手发的一条推文。”从 Cursor 的角度来看，这更像是一场无解的公关神话，即使按传闻中每月花掉几百万美元，其产生的产品宣传和流量回报也完全超值。

Lauren 本人也曾在访谈中回应了成本问题，承认了实验室特权对她方法论的绝对依赖：“我在 AI 实验室工作，token 不限量，所以我不能说每个人都应该完全照着我的方法做。”但她同时将这套逻辑定性为一种前期的投入产出比博弈：前期花 Token 重构、加约束，是为了让最基础的 Agent 也能写好代码，从而让产品经理和设计师直接交付功能。“Agent 的价值在于让你做以前做不到的事，不是把 Token 丢到每一个小问题上。”

尽管她坚信“不必花到倾家荡产，也有机会达到类似的状态”，但这依然无法掩盖一个残酷的技术现实：没有实验室无限 Token 的底座支持，高频的自动化验证闭环根本无从谈起，这套方法论对成本极其敏感的普通客户和用户而言，高昂的验证开销意味着难以跨越的门槛。

本文来源：InfoQ

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

限时权益申领 * 体验资格每人限申领一次
VIP会员·7天畅读卡 | 大师课·入门串讲
扫码 免费领取

限免

93 收藏 分享到:

写评论

请发表您的评论

表情 图片

发布评论